Chapitre 1 : Introduction à la Programmation Orientée Objet en Java

L'objectif de cette première fiche est de présenter quelques éléments généraux liés à la programmation orientée objet et au langage Java.

Programmation Orientée Objet

La Programmation Orientée Objet (POO), ou *Object Oriented Programming* (OOP) en anglais, est un **paradigme** de programmation. Tout programme est structuré autour d'**objets** communicant entre eux, chaque objet représentant une entité du problème traité, concrète ou abstraite (par exemple une voiture ou un ensemble de données). Un objet est défini par son état (les données qu'il représente) et ses comportements (les actions qui peuvent lui être appliquées). Quelques notions fondamentales, présentées dans la suite de ce cours sont les classes et objets, l'encapsulation, l'héritage et le polymorphisme.

Le paradigme OO a été introduit dans les années 1970, puis intégré (en tout ou partie) dans différents langages de programmation: SmallTalk (1972), Eiffel, C++ (84), Java (91), Ada (95), C# , Python...

Il est aussi important de mentionner l'existence de langages de modélisation OO, en particulier *Unified Modeling Langage* (UML), permettant de concevoir une structure OO de manière générique, indépendamment d'un langage de programmation particulier. Ces notions de modélisation OO ne seront que très peu abordées dans ce cours, mais seront couvertes ultérieurement dans d'autres enseignements.

Le langage Java

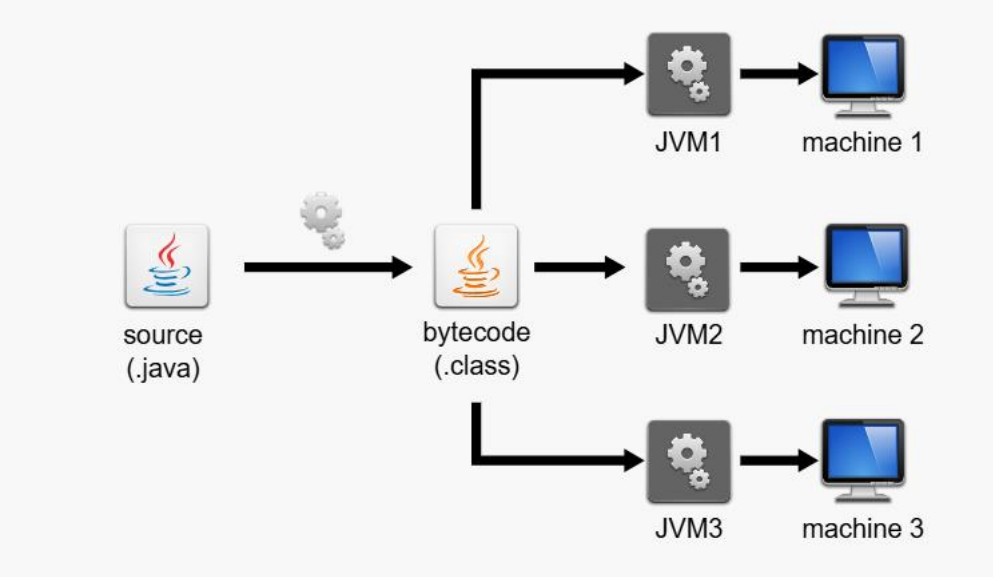
Le langage Java a été créé par James Gosling, Patrick Naughton et Bill Joy en 1995. Il était initialement supporté par la société Sun Microsystems, depuis rachetée par Oracle. Des implémentations publiques du langage existent toutefois, en particulier OpenJDK.

La syntaxe Java est (volontairement) très proche de celle du C/C++, mais c'est un langage purement Orienté Objet (contrairement au C++ par exemple, qui permet de mélanger les paradigmes impératif et objet). Il est extrêmement répandu notamment pour des applications client-serveur et web (*applets*). En plus de sa simplicité, une de ses grandes forces est sa **portabilité**: c'est en fait un langage interprété par une *Machine Virtuelle Java* (JVM), et un même programme peut donc être exécuté sur toute architecture dotée d'une JVM.

La chaîne de compilation et d'exécution Java

Java est à la fois un langage *compilé* et un langage *interprété*. Il est compilé, car le code source (les fichiers `.java`) n'est pas directement exécutable, mais doit être compilé au préalable. La compilation produit ce que l'on appelle le *bytecode* Java (code binaire), sous la forme de fichiers `.class`. Ces fichiers ne sont pas non plus directement exécutables par une machine, mais le sont par une *machine virtuelle Java* (JVM). La JVM est donc une application qui est chargée d'interpréter le code binaire Java et d'exécuter le programme correspondant sur la machine.

La chaîne de compilation est résumée dans la figure ci-dessous. Comme on peut le voir sur la figure, l'un des grands avantages de l'interprétation est d'avoir un code binaire générique qui est le même pour toutes les architectures. Ainsi, nul besoin de recompiler votre programme pour une architecture particulière : si la machine possède une JVM, alors votre code binaire sera directement exécutable sur cette machine.



Compiler un programme java

Pour compiler un programme java, il faut utiliser le compilateur `javac`. Il faut passer un ou plusieurs noms de fichiers sources `.java` en argument de la ligne de commande au lancement du compilateur. Le compilateur est capable de faire tout seul l'édition de liens (d'aller chercher les ressources dont il a besoin). Ainsi, par exemple, la commande :

```
javac TestPangolin.java
```

compilera non seulement `TestPangolin.java`, mais également toutes les classes qui pourraient être nécessaires à son exécution (`Pangolin.java` par exemple).

Exécuter un programme Java

Comme nous l'avons vu, le code binaire n'est pas directement interprétable par la machine, mais il l'est par une *machine virtuelle Java* (pour cela, il faut l'avoir installé sur votre machine). Grâce à cette machine virtuelle, on peut lancer le programme.

```
java TestPangolin
# Attention, cette commande est à lancer sur le nom de la classe, pas sur le nom du fichier
# (notez l'absence de .class)
```

L'API standard Java

L'Application Programming Interface (API) standard Java regroupe les classes natives du langage en un ensemble normalisé de fonctionnalités. Elle est maintenue avec le langage.

L'API Java est remarquablement bien fournie : elle comprend par exemple des collections, des classes regroupant des algorithmes courants tout prêt à l'usage, des classes pour les entrées/sorties, le réseau, l'IHM, la programmation multi-thread, etc.

La **référence essentielle** pour trouver les classes existantes et leur spécification (leurs attributs, méthodes et constructeurs) est la documentation de cette API Application Programming Interface (API) standard Java. Cette documentation devrait rapidement devenir l'un de vos grands amis !

La documentation Javadoc

Note

Les notions évoquées ici (classes, méthodes, etc.) n'ont peut-être pas encore de sens si vous découvrez le langage via cette première fiche. Allez lire la fiche classes et objets avant de revenir ici, et tout sera clair.

Dans certains langages comme C ou ADA par exemple, on trouve à trois endroits différents :

- la spécification (`.ads`, `.h`) ;
- le code (`.adb`, `.c`) ;
- la documentation

En Java, tout est décrit au même endroit dans un fichier `.java`. C'est lors de la phase de compilation que les choses sont clairement séparées. Cela implique que la documentation des programmes (à destination des développeurs qui vont réutiliser vos classes) doit se faire dans le code-même.

En fait, le principe est très simple. Chaque élément du code (classe, attribut, méthode) est documenté en utilisant des commentaires spécifiques précédent la déclaration de l'élément). Ces commentaires sont entourés par `/**` et `*/`, et peuvent contenir des mots-clefs spéciaux interprétés par le compilateur de documentation (par exemple `@author`, `@version`, `@param`, `@return`...).

Exemple sur la classe Pangolin :

```
/**
 * Cette classe fournit l'implantation d'un Pangolin,
 * petit mammifère d'Asie et d'Afrique ressemblant
 * « à un artichaut à l'envers avec des pattes,
 * prolongé d'une queue à la vue de laquelle on se
 * prend à penser qu'en effet, le ridicule ne tue plus. »
 *
 * @author Sylvain B.
 * @version 1.0
 */
public class Pangolin {
    /** la coordonnée x du pangolin. */
    private double x;
    /** la coordonnée y du pangolin. */
    private double y;
    /** le nom du pangolin (traditionnellement Gérard ou Toto). */
    private String name;
    /** le nombre d'écailles du pangolin (doit être strictement positif).
*/
    private int nbEcailles;

    /**
     * Translate un pangolin.
     *
     * @param dx la coordonnée x de translation
     * @param dy la coordonnée y de translation
```

```
     */
    public void translater(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    /**
     * Cette méthode émet le cri caractéristique du pangolin
     * et l'affiche sur la sortie standard.
     */
    public void crier() {
        System.out.println("Gwwark Rhââgn Bwwikk"); // Cri du pangolin
    }
}
```

L'utilitaire `javadoc` permet de compiler la documentation sous forme de fichiers HTML à partir d'un ensemble de fichiers sources.

Par exemple, pour générer la documentation du fichier `Pangolin.java` précédent, on utilisera dans le Terminal :

```
javadoc Pangolin.java

# Pour générer la documentation de toutes les classes d'un répertoire :
javadoc _le_répertoire_

# Pour préciser dans quel répertoire générer la documentation html,
utiliser l'option -d :
javadoc Pangolin.java -d documentation
```

La documentation générée par `javadoc` est présentée sous la même forme standard que la documentation de l'API standard Java D'ailleurs, celle-ci a bien sûr été générée avec l'utilitaire `javadoc`.

Un mot sur les IDE (Integrated Development Environment)

Bien entendu, vous pouvez écrire vos fichiers `.java` avec votre éditeur de texte préféré, puis compiler avec `javac` et exécuter avec `java`. Ceci devient parfois limité dans un projet de plus grande ampleur (en nombre de fichiers ou nombre de développeurs) ou en cas de bugs.

Des outils intéressants sont les *environnements de développement intégrés* (IDE), applications qui regroupent un ensemble d'outils pour la programmation. Par exemple:

- Un éditeur de texte
- La complétion sur les noms de classes ou de méthodes, et accès direct à la documentation
- Génération de certaines méthodes (set/get, equals...), à la demande
- Import/export UML
- La compilation à la volée, pour voir tout de suite vos éventuelles erreurs de syntaxe
- Exécution

- Débogueur intégré
- Gestion de projet intégrée (via GIT, SVN, ...)
- Et bien d'autre choses encore...

Il existe énormément d'IDE, pour tous les langages d'ailleurs. Pour Java en particulier, deux des plus utilisés sont Eclipse et NetBeans.

Dans ce module, nous utiliserons :

- Un éditeur de texte standard, et une compilation / exécution à la ligne de commande, pour les deux premières séances. Il est fondamental que vous sachiez vous débrouiller sans IDE, ne serait-ce que pour comprendre comment paramétrer correctement la compilation et l'exécution de vos programmes dans votre IDE préféré.
- Eclipse à partir de la 3ème séance. Vous aurez une petit TD d'introduction à Eclipse dès cette séance-là.